Network Diagnostic and Path Analysis Tool

Overview

This project implements a UDP-based network diagnostic tool to measure round-trip time (RTT) and packet loss in a network created using Mininet. It also includes a diamond-shaped network topology with alternative paths and uses a Ryu SDN controller.

The purpose is to understand network performance through application-level probes and explore how network topology affects communication.

Objectives

- Send UDP diagnostic probes between network hosts.

- Measure round-trip time for successful probes.

- Detect packet loss using response timeouts.

- Create a Mininet diamond topology with alternative routes.

- Use a Ryu controller to manage the software-defined network.

Technologies Used

- Python 3 — diagnostic client and server.

- UDP sockets — sending probes and receiving acknowledgements.

- Mininet — virtual network topology.

- Open vSwitch— virtual network switches.

- Ryu — SDN controller.

Project Structure

- `client.py` — sends UDP probes and calculates RTT and packet loss.

- `server.py` — receives probes and sends acknowledgements.

- `topo.py` — defines the Mininet diamond topology.

- `ryu/` — Ryu controller-related files.

- `terminal1_commands/` — commands for the first terminal.

- `terminal2_commands/` — commands for the second terminal.

Requirements

The project is intended to run on Ubuntu Linux with Python 3, Mininet, Open vSwitch and Ryu installed.

Install the system packages using:

sudo apt update

sudo apt install mininet openvswitch-switch python3-venv python3-full python3-pip git -y

Create a virtual environment for Ryu:

python3 -m venv ~/ryu_env

source ~/ryu_env/bin/activate

pip install setuptools==69.5.1

pip install ryu

How to Run

1. Clone the repository:

   git clone https://github.com/Vishy01K/cn_project.git

   cd cn_project

2. Open the `terminal1_commands/` and `terminal2_commands/` folders and follow the commands provided there in the specified order.

3. Activate the Ryu virtual environment in the terminal used for the controller:

   source ~/ryu_env/bin/activate

4. Start the Ryu controller and Mininet topology according to the provided terminal instructions.

5. In the appropriate Mininet host terminals, start the UDP server:

   python3 server.py

6. From the client host, run the diagnostic client, replacing the IP with the server host's actual IP address:

python3 client.py 10.0.0.2

The client sends 10 UDP probes to port 5005. Successful replies produce RTT measurements, while timeouts are counted as packet loss.

Note: The controller startup command, topology startup command, and host file paths must match the actual instructions in the repository. Verify these steps on the target Ubuntu environment before using this README as the final setup guide.

Expected Output

The client reports the result of each probe, including its success or timeout, followed by the total number of probes sent, the number lost, and the packet-loss percentage.

Limitations

The current diagnostic client measures application-level RTT and packet loss. Automatic topology discovery, correlation of measurements with individual network paths, and automatic comparison of alternative paths require corresponding controller and path-analysis functionality.

Repository

All the required commands are attached in the repository

[https://github.com/Vishy01K/cn_project]